

**Московский авиационный институт
(национальный исследовательский университет)**

**Факультет информационных технологий и прикладной
математики**

Кафедра вычислительной математики и программирования

Лабораторная работа №1 по курсу «Дискретный анализ»

Студент: М. А. Хорохорин
Преподаватель: А. А. Кухтичев
Группа: М8О-206БВ-24
Дата: 05.03.2026
Оценка:
Подпись:

Москва, 2026

Лабораторная работа №1

Задача: Требуется разработать программу, осуществляющую ввод пар «ключ-значение», их упорядочивание по возрастанию ключа указанным алгоритмом сортировки за линейное время и вывод отсортированной последовательности.

Вариант сортировки: Поразрядная сортировка.

Вариант ключа: даты в формате DD.MM.YYYY, например 1.1.1, 01.09.2009, 31.12.2009.

Вариант значения: строки фиксированной длины 64 символа. Входные строки могут быть короче и дополняются нулями до 64 символов, которые не выводятся.

1 Описание

Требуется написать реализацию алгоритма поразрядной сортировки для пар «дата-значение».

Основная идея поразрядной сортировки заключается в том, чтобы каждый входной элемент x разбить на разряды и сортировать стабильной сортировкой по каждому разряду.

2 Исходный код

На каждой непустой строке входного файла располагается пара «ключ-значение», поэтому создадим новую структуру *Pair*, в которой будем хранить исходную строку и преобразованный в `uint32_t` ключ.

Ключом является дата в формате DD.MM.YYYY, которая преобразуется в числовое значение по формуле $year \times 10000 + month \times 100 + day$. Такое представление позволяет корректно сравнивать даты как целые числа. Исходная строка хранится для восстановления вывода, а числовой ключ используется исключительно для сортировки.

Так как заранее неизвестно количество входных строк, необходимо реализовать динамический массив *Vector*. Для него реализуется правило пяти (деструктор, конструктор копирования, оператор копирования, конструктор перемещения, оператор перемещения), а также дополнительные функции: `PushBack`, `Size` и вывод в поток.

Далее необходимо реализовать саму поразрядную сортировку `RadixSort`. Для неё требуется быстрая стабильная сортировка — в данной работе используется сортировка подсчётом по отдельному байту числа `CountingPass`.

Так как ключ хранится в типе данных `uint32_t`, он занимает 4 байта, что соответствует 4 разрядам. Для каждого разряда выделяется дополнительный массив размером 256, так как в одном байте содержится 8 бит, а $2^8 = 256$.

Индекс массива, в который необходимо увеличить значение счётчика, получается побитовым сдвигом ключа на нужное количество бит и побитовой операцией И с маской `0xFF`, чтобы получить последние 8 бит.

Остальная часть алгоритма представляет собой стандартную реализацию сортировки подсчётом: строится массив частот, затем преобразуется в массив префиксных сумм, после чего элементы расставляются в выходной массив в обратном порядке для обеспечения стабильности.

Также используется двойной буфер для сортировки: вместо выделения нового массива на каждом шаге алгоритма используется заранее выделенный буфер `buffer`, а результат каждого прохода перемещается обратно в исходный массив.

main.cpp	
void CountingPass(Vector<Pair>& input, Vector<Pair>& output, int byteIndex)	Функция одного прохода сортировки подсчётом по заданному байту ключа
main.cpp	
void RadixSort(Vector<Pair>& data)	Функция поразрядной сортировки

```

1  template <typename T>
2  class Vector {
3  private:
4      T* data;
5      size_t size;
6      size_t capacity;
7
8      void Reallocate(size_t newCapacity);
9
10 public:
11     Vector();
12
13     Vector(size_t n, T value = T{});
14
15     Vector(const Vector& other);
16
17     Vector& operator=(const Vector& other);
18
19     Vector(Vector&& other) noexcept;
20
21     Vector& operator=(Vector&& other) noexcept;
22
23     ~Vector();
24
25     size_t Size() const;
26
27     void PushBack(T&& value);
28
29     T& operator[](size_t i);
30
31     const T& operator[](size_t i) const;
32
33     friend std::ostream& operator<<(std::ostream& os, const Vector& v);
34 };
35
36 struct Pair {
37     std::string raw;
38     uint32_t key;
39
40     Pair();
41

```

```

42 |     explicit Pair(const std::string& line);
43 |
44 |     Pair(const Pair& other);
45 |
46 |     Pair& operator=(const Pair& other);
47 |
48 |     Pair(Pair&& other) noexcept;
49 |
50 |     Pair& operator=(Pair&& other) noexcept;
51 |
52 |     ~Pair() = default;
53 |
54 |     friend std::ostream& operator<<(std::ostream& os, const Pair& p);
55 | };

```

3 Консоль

```
root@host:/workspaces/lab# cmake -B build
root@host:/workspaces/lab# cmake --build build
root@host:/workspaces/lab# ./build/lab/lab
01.01.2000 Alice
15.06.1995 Bob
03.03.2010 Charlie
20.11.1988 Diana
01.01.2000 Alice
15.06.1995 Bob
20.11.1988 Diana
03.03.2010 Charlie
```

4 Тест производительности

Тест производительности представляет из себя следующее: создаётся 2 одинаковых массива, один сортируется с помощью алгоритма, который мы реализовали, а другой — с помощью стандартной функции `std::stable_sort`. Время выполнения обоих алгоритмов замеряется и сравнивается.

```
root@794a05b215ca:/workspaces/discrete-analysis-labs# python3 lab1/benchmark/generator
100
root@794a05b215ca:/workspaces/discrete-analysis-labs# ./build/lab1/lab1_benchmark
<tests.txt
Radix sort time: 108us
STL stable sort time: 150us
root@794a05b215ca:/workspaces/discrete-analysis-labs# python3 lab1/benchmark/generator
1000
root@794a05b215ca:/workspaces/discrete-analysis-labs# ./build/lab1/lab1_benchmark
<tests.txt
Radix sort time: 1007us
STL stable sort time: 1258us
root@794a05b215ca:/workspaces/discrete-analysis-labs# python3 lab1/benchmark/generator
10000
root@794a05b215ca:/workspaces/discrete-analysis-labs# ./build/lab1/lab1_benchmark
<tests.txt
Radix sort time: 8856us
STL stable sort time: 14353us
root@794a05b215ca:/workspaces/discrete-analysis-labs# python3 lab1/benchmark/generator
100000
root@794a05b215ca:/workspaces/discrete-analysis-labs# ./build/lab1/lab1_benchmark
<tests.txt
Radix sort time: 113872us
STL stable sort time: 175208us
root@794a05b215ca:/workspaces/discrete-analysis-labs# python3 lab1/benchmark/generator
1000000
root@794a05b215ca:/workspaces/discrete-analysis-labs# ./build/lab1/lab1_benchmark
<tests.txt
Radix sort time: 833755us
STL stable sort time: 1627027us
```

Как видно из результатов, поразрядная сортировка опережает `std::stable_sort` на всех размерах входных данных. Это объясняется тем, что асимптотическая сложность нашего алгоритма составляет $O(n \cdot k)$, где $k = 4$ (количество байт в `uint32_t`)

— то есть фактически $O(n)$, тогда как сложность `std::stable_sort` равна $O(n \log n)$. При увеличении числа элементов с 100 до 1 000 000 разрыв становится всё более заметным: на миллионе элементов наш алгоритм работает почти в **2 раза быстрее**.

5 Выводы

Выполнив данную лабораторную работу по курсу «Дискретный анализ», я научился реализовывать алгоритм поразрядной сортировки и сортировки подсчётом. Разобрался, как сортировать данные побайтово, используя тип `uint32_t` в качестве ключа, а также как компактно кодировать дату в числовое значение для эффективного сравнения. Вспомнил, как писать шаблонный динамический массив и применять правило пяти.

Список литературы

- [1] Томас Х. Кормен, Чарльз И. Лейзерсон, Рональд Л. Ривест, Клиффорд Штайн. *Алгоритмы: построение и анализ, 2-е издание*. — Издательский дом «Вильямс», 2007. Перевод с английского: И. В. Красиков, Н. А. Орехова, В. Н. Романов. — 1296 с. (ISBN 5-8459-0857-4 (рус.))
- [2] *Сортировка подсчётом* — *Википедия*.
URL: https://ru.wikipedia.org/wiki/Поразрядная_сортировка (дата обращения: 16.12.2013).
- [3]